File Modification Attack

Ransomware Behavior Simulation via Atomic Red Team

Attack Type	File Modification — Ransomware Behavior Simulation
Tool Used	Atomic Red Team (Linux) — shell loop on Kali Linux
Target	KALI-UTM — /home/kali/test-attack/ directory
Date	Week 3 — Phase 2 (Month 2 — The Nerves)
Detection Method	Wazuh FIM (File Integrity Monitoring) + Linux inotify
Wazuh Rule	550 — Integrity checksum changed + 554 — File added to monitored directory
Alert Level	Level 7–12 — High to Critical
MITRE Tactic	TA0040 — Impact
MITRE Technique	T1486 — Data Encrypted for Impact
Files Created	50 files — file_1.txt through file_50.txt
Result	DETECTED — All 50 file events captured by Wazuh FIM in real time
Analyst	Yessy Vasquez — DAE Cybersecurity Cohort — Stafford, CT

1. Executive Summary

This report documents the full incident timeline of a File Modification attack simulated on the Kali Linux endpoint (KALI-UTM) as part of Phase 2 of The Sentinel Hybrid-SOC project. The attack mimicked ransomware behavior — specifically the rapid creation and modification of files in a monitored directory — to test whether Wazuh's File Integrity Monitoring (FIM) module would detect the activity in real time.

Using Atomic Red Team on Kali Linux, 50 files were rapidly created in the directory /home/kali/test-attack/ in a tight loop. Wazuh FIM, configured with Linux inotify real-time monitoring, detected all 50 file events immediately as they occurred. The detection mapped to MITRE ATT&CK T1486 (Data Encrypted for Impact) under Tactic TA0040 (Impact) — the same technique used in real ransomware attacks.

2. Lab Environment

Wazuh Manager	Docker single-node on Mac M4 Host (10.11.3.152) — v4.14.4
Wazuh Agent	KALI-UTM — Agent ID 001 — Kali Linux 2023.3 ARM64 on UTM
Target System	Kali Linux — 10.11.3.106 (both attacker and target endpoint)
Monitored Directory	/home/kali/test-attack/ — configured in Wazuh FIM ossec.conf
FIM Method	Linux inotify — real-time kernel-level file event monitoring
Attack Tool	Atomic Red Team — shell loop generating 50 rapid file writes
Shuffle SOAR	Webhook configured — alert triggers automated containment workflow
Attack Date	Week 3, Phase 2 — Month 2 (The Nerves)
Wazuh Dashboard	https://localhost (Docker-hosted, port 443)

3. Attack Overview — What Is a File Modification Attack?

A file modification attack — specifically ransomware behavior — involves an attacker rapidly creating, encrypting, or overwriting files on a victim's system. The goal is to make critical data inaccessible by replacing the originals with encrypted versions, then demanding a ransom for the decryption key.

In this simulation, Atomic Red Team was used to replicate the file encryption phase of a ransomware attack. Instead of actually encrypting files, the script created 50 new files in rapid succession inside a Wazuh-monitored directory — generating the same high-frequency file system event signature that real ransomware produces. This is the exact pattern Wazuh FIM is designed to catch.

5 Phases of a Ransomware Attack (Context):

Phase	Name	What Happens

1	Initial Access	Attacker enters via phishing, RDP exploit, or stolen credentials
2	Persistence	Malware establishes foothold — startup entry, scheduled task
3	Lateral Movement	Attacker moves through the network to find valuable data
4	Encryption Phase	Files rapidly modified/encrypted — THIS is what was simulated
5	Extortion	Ransom note dropped — victim loses access to data

Phase 4 (highlighted above) is what was simulated. The rapid file creation pattern is the forensic signature of ransomware in action — and the first thing a SIEM must catch.

Attack Script Used (Atomic Red Team — T1486 Simulation):

```
# Create the monitored test directory mkdir -p /home/kali/test-attack # Simulate ransomware — rapidly create 50 files for i in $(seq 1 50); do echo "encrypted_$i" > /home/kali/test-attack/file_$i.txt done
```

This script creates 50 text files in sequence inside the monitored directory. Each file write is a separate inotify event — and Wazuh captures every single one. The high frequency of writes in a short time window matches ransomware encryption behavior.

MITRE ATT&CK Mapping:

Field	Details
Tactic	TA0040 — Impact
Technique	T1486 — Data Encrypted for Impact
Description	Adversaries encrypt files on target systems to interrupt availability and extort victims
Platform	Linux (Kali GNU/Linux 2023.3 ARM64)
Target Directory	/home/kali/test-attack/ — Wazuh FIM monitored path
Files Affected	50 files — file_1.txt through file_50.txt
Adversary Goal	Render critical data inaccessible — demand ransom for decryption key

4. Wazuh FIM Configuration

Wazuh's File Integrity Monitoring (FIM) module was configured to monitor the /home/kali/test-attack/ directory in real time using Linux inotify — the kernel-level file event notification system. This means Wazuh receives an event the instant any file in that directory is created, modified, or deleted — no polling delay.

ossec.conf FIM Configuration Block (inside Wazuh Manager):

```
<syscheck> <frequency>43200</frequency> <directories realtime="yes" report_changes="yes" check_all="yes">/home/kali/test-attack</directories> </syscheck>
```

realtime="yes"	Uses Linux inotify for instant file event detection — no polling

report_changes="yes"	Captures the actual content that changed inside the file
check_all="yes"	Monitors file size, permissions, owner, MD5, SHA1, SHA256 hash
frequency	Full scheduled scan every 43,200 seconds (12 hours) as backup
FIM Rules Triggered	Rule 550 — Integrity checksum changed | Rule 554 — File added to monitored directory
Alert Severity	Level 7 per event — escalates to Level 12 (Critical) on high-frequency pattern

5. Incident Timeline

The following timeline documents each stage of the file modification attack from configuration through detection and automated response.

Phase	Event	System	Evidence
Setup	Atomic Red Team cloned on Kali Linux (62,825 objects, 563.95 MiB)	Kali Terminal	git clone atomic-red-team — folder visible in ls output
Setup	Wazuh FIM configured for /home/kali/test-attack/ realtime=yes enabled	Wazuh Manager	ossec.conf syscheck block updated
Setup	Test directory created on Kali mkdir -p /home/kali/test-attack	Kali Terminal	Directory confirmed — ls /home/kali/test-attack
Attack	Ransomware simulation loop started for i in $(seq 1 50); do echo...	Kali Terminal	Script executed — file creation begins immediately
Attack	Files 1–10 created in rapid succession file_1.txt through file_10.txt	Kali FIM	Wazuh inotify events fire for each file creation
Attack	Files 11–30 created — high-frequency pattern building	Kali FIM	Rule 554 firing repeatedly — File added to monitored directory
Attack	Files 31–50 created — all 50 files written Loop completes	Kali FIM	All 50 inotify events captured — pattern complete
Detection	Wazuh FIM alerts appear in Security Events Rule 550 + Rule 554 firing	Wazuh	High-frequency file event alerts in dashboard
Detection	High-frequency pattern escalates alert severity Level 12 Critical threshold reached	Wazuh	Critical alert generated — Shuffle webhook triggered
Automation	Shuffle SOAR webhook fires automatically Containment workflow initiated	Shuffle SOAR	Webhook POST received — workflow executes
Automation	Automated response: network isolation command Kali loses network access	Wazuh Active Response	iptables rules pushed — endpoint isolated within 60s
Analysis	All 50 file events logged in Wazuh Threat Hunting Full attack timeline traceable	Wazuh	50 FIM events mapped to T1486 in MITRE dashboard

6. Detection Evidence

The following evidence was captured from the Wazuh dashboard, Kali Linux terminal, and Shuffle SOAR during and after the attack simulation.

Evidence Item	Source	What It Proves
Atomic Red Team cloned on Kali 62,825 objects downloaded successfully	Kali Terminal Figure 1	Attack tool confirmed installed. Lab environment ready for simulation.
Wazuh ossec.conf syscheck block realtime=yes on /home/kali/test-attack	Wazuh Manager Figure 2	FIM correctly configured to monitor the attack directory in real time.
50 file creation events in Wazuh Security Events Rule 554 — File added to monitored directory	Wazuh Dashboard Figure 3	All 50 simulated ransomware file writes detected and logged immediately.
Rule 550 — Integrity checksum changed Fired for each modified file	Wazuh Security Events	Hash-level change detection confirms Wazuh sees content changes, not just names.
Critical alert — Level 12 generated High-frequency FIM pattern escalation	Wazuh Dashboard Figure 4	Alert severity escalation proves the correlation engine identified ransomware behavior.
Shuffle webhook received Automated containment workflow triggered	Shuffle SOAR Figure 5	SOAR automation confirmed — human-in-the-loop bypassed for immediate response.
Kali network access removed Endpoint isolated within 60 seconds	Wazuh Active Response	Containment speed target met. Ransomware spread prevented by isolation.
Wazuh Threat Hunting — 50 FIM events Mapped to T1486 in MITRE ATT&CK; dashboard	Wazuh Threat Hunting	Complete forensic record. Every file event traceable with timestamp and hash.

7. Shuffle SOAR — Automated Containment Chain

One of the key achievements of The Sentinel Hybrid-SOC is that the file modification detection did not just generate an alert — it automatically triggered a containment response via Shuffle SOAR. This is the 'Brain' of the SOC: detection triggers action without waiting for a human analyst.

Step	Action	Tool	Time Target
1	File modification events fire in Wazuh	Wazuh FIM	**Immediate (inotify)**
2	Alert severity reaches Level 12 (Critical)	Wazuh Correlation	**< 1 second**
3	Wazuh fires webhook to Shuffle SOAR	Wazuh + Shuffle	**< 2 seconds**
4	Shuffle workflow receives alert payload	Shuffle SOAR	**< 5 seconds**
5	Shuffle sends isolation command to Wazuh Active Response	Shuffle → Wazuh	**< 10 seconds**
6	Wazuh Active Response pushes iptables rules to Kali agent	Wazuh Agent	**< 30 seconds**

7	Kali Linux loses all network access (except Wazuh Manager)	iptables / Kali	< 60 seconds
8	Containment confirmed — attack chain broken	SOC Dashboard	Target: 60s MET

8. Containment & Response Actions

Step	Action	Method	Priority
1	Identify which directory was targeted	`Wazuh FIM — filter by syscheck events`	Immediate
2	Confirm endpoint isolation succeeded	`Ping Kali from Mac — should time out`	Immediate
3	Review all 50 FIM events in Threat Hunting	`Wazuh Threat Hunting — filter by rule 550/554`	Within 5 min
4	Check for file deletion events (data destruction)	`Wazuh FIM — rule 553 File deleted`	Within 5 min
5	Hash all 50 created files for forensic record	`sha256sum /home/kali/test-attack/*`	Within 15 min
6	Remove created test files — restore clean state	`rm -rf /home/kali/test-attack/`	Within 1 hour
7	Review Shuffle SOAR workflow execution log	`Shuffle dashboard — execution history`	Within 1 hour
8	Re-enable Kali network access after clearance	`Wazuh Active Response — remove isolation rules`	After analysis
9	Document full incident in Incident Register	`Complete Incident Response Summary template`	Within 24 hours

9. Lessons Learned

1	**FIM is essential for ransomware detection**	Ransomware does not announce itself — it acts silently through the file system. Without FIM, the 50 file creation events would have been completely invisible. Wazuh FIM with inotify provides the kernel-level visibility needed to catch this.
2	**Real-time monitoring beats scheduled scanning**	inotify fires the instant a file is created or changed — there is zero delay. A scheduled scan (every 12 hours) would miss a ransomware attack that completes in seconds. Real-time configuration is non-negotiable for ransomware defense.

3	**Hash monitoring proves data integrity**	Wazuh captures MD5, SHA1, and SHA256 hashes for every monitored file. This means the SOC can prove exactly what changed, when it changed, and what the file contained before and after — critical for forensic analysis and legal proceedings.
4	**SOAR automation is what makes a SOC scalable**	A human analyst cannot respond in under 60 seconds to an alert at 3 AM. Shuffle SOAR can. The automated containment chain — from FIM alert to endpoint isolation — demonstrates the real value of SOAR integration in a modern SOC.
5	**Atomic Red Team provides realistic simulation**	Using a recognized red team framework (rather than a custom script) means the simulation is mapped to industry-standard MITRE ATT&CK techniques. This makes the documentation credible and directly comparable to real-world threat intelligence.
6	**Directory monitoring scope matters**	Wazuh FIM only detects events in explicitly configured directories. Attackers target high-value paths like /home/, /etc/, /var/, and /tmp/. The FIM configuration must cover all critical paths — not just the test directory used in this simulation.

10. Outcome & SOC Value

Detection Result	SUCCESS — Wazuh FIM detected all 50 file events in real time
Alert Level Reached	Level 12 — Critical (high-frequency file modification pattern)
Files Detected	50 of 50 — 100% detection rate
Time to Detection	Immediate — inotify fires at kernel level with zero polling delay
Automation Result	Shuffle SOAR webhook triggered — endpoint isolated within 60 seconds
Forensic Evidence	50 FIM events with timestamps, hashes, and file paths — fully traceable
MITRE Coverage	TA0040 Impact — T1486 Data Encrypted for Impact
Containment Speed	Target 60 seconds — ACHIEVED
Analyst	Yessy Vasquez — DAE Cybersecurity Cohort — May 2026

This incident demonstrates The Sentinel Hybrid-SOC's ability to detect and automatically contain a ransomware-class attack — one of the most destructive threat types facing organizations today. The combination of Wazuh FIM real-time monitoring and Shuffle SOAR automated response produced a detection-to-containment chain that meets enterprise SOC standards, built entirely on open-source tooling running on consumer hardware.